Legenda. Prioridade: **P0** bloqueia o resto, **P1** alta, **P2** média, **P3** baixa. Quando o documento não diz algo explicitamente, marco como "inferência". As etapas seguem as sprints do §8 do documento. Não estimo horas ou dias, porque não sei o tamanho real do time, a disponibilidade nem se já existe código.

Parte 1 — Resumo executivo

O documento é uma boa **proposta de arquitetura**, mas ainda não é um **plano executável para o Front-end**.

Pontos positivos

- Stack coerente com o escopo (React + Vite + TS, Spring Boot, PostgreSQL).
- Monorepo e modelo de dados simples, com `Recurso` único para ambientes e equipamentos.
- Divisão clara entre MVP e Fase 2.
- Fluxo Git definido e sprints mapeadas.

Principais problemas

1. A divisão do grupo é só um "exemplo", e não diz qual Front-end você é.
2. O escopo do MVP contradiz a Fase 2 em três pontos: aprovação/PENDENTE, notificações e fotos.
3. O coworking tem "capacidade", mas a *exclusion constraint* permite só uma reserva por vez por recurso.
4. Não há contrato de API, requisitos, critérios de aceite, wireframes nem designer.
5. A fundação do front (estrutura, rotas, login, layout, API, Docker, CI, deploy) e os módulos `notificacao`, `relatorio` e `common` estão sem dono.

O que mais merece atenção agora: confirmar seu papel, a stack e o repositório. Em seguida, o contrato de auth/recursos, o coworking, as datas/fuso e o fechamento do escopo do MVP.

Parte 2 — Diagnóstico do documento

Tipo: *Problema real* (lacuna ou contradição), *Melhoria* ou *Confirmar* (decisão da equipe).

ID	Seção	Tipo	O que foi identificado e por que é um problema	Recomendação	Quando resolver

D1	§6	Confirmar	A divisão é "exemplo com 5 integrantes". Não diz quem é Front-end 1 ou 2, nem se o grupo tem 5 pessoas. Risco de trabalho duplicado ou áreas descobertas.	Confirmar o time real e o seu recorte. Se você for o único FE, assume as duas funções (o §6 prevê essa fusão).	Antes de codar
D2	§2	Confirmar	A stack é "sugerida" e o texto admite trocá-la. Também não define libs do front (rotas, UI/estilo, dados do servidor, formulários, calendário, testes).	Confirmar React + Vite + TS e fechar um conjunto mínimo de libs. Evitar Redux e similares: para este escopo, Context só para auth e uma lib de dados do servidor bastam.	Antes de codar
D3	§4, §5	Problema real	Contradições MVP × Fase 2. `PENDENTE` e "exige aprovação" existem no modelo, mas "fluxo de aprovação" e "regras avançadas" são Fase 2. `NAO_COMPARECEU` está no ciclo, mas "penalidade" é Fase 2. Não está claro se a reserva nasce PENDENTE ou CONFIRMADA, nem quais regras de uso valem no MVP.	O PO fecha a lista exata de status, regras e campos do MVP.	Antes da Sprint 2
D4	§2, §4, §5	Problema real	E-mail aparece na arquitetura, no modelo (`Notificacao`) e no repositório, mas lembretes são Fase 2. `Emprestimo` tem "observações/fotos", mas fotos são Fase 2. Risco de construir tela desnecessária ou deixar de construir uma necessária.	Declarar notificações e fotos fora do MVP (ou o contrário). No front, só campo de texto para condição e observações.	Antes da Sprint 3

D5	§1, §4	Problema real (provável)	Coworking é ambiente compartilhado com "capacidade", mas a constraint impede duas reservas sobrepostas no mesmo recurso. Na prática, o coworking aceitaria uma reserva por vez. Calendário e formulário dependem dessa definição.	Decidir: reservar por posto (cada posto é um Recurso), por capacidade (exige regra nova) ou o espaço inteiro.	Antes da Sprint 2
D6	§4	Problema real	Reserva tem início/fim, mas nada define fuso, formato, granularidade (30 min? 1 h?), horário de funcionamento ou reserva de vários dias (provável em equipamentos). Também não define o que a consulta de disponibilidade devolve.	Definir formato e fuso das datas (ex.: ISO 8601 com offset), granularidade e o formato da resposta de disponibilidade.	Antes da Sprint 2
D7	§2, §5	Problema real	JWT sem definição de armazenamento, expiração/refresh, logout, tratamento de 401/403 e criação do primeiro ADMIN. "Cadastro de profissional e admin" pode implicar auto-cadastro de admin, o que seria falha de segurança. Guardas de rota no front são só UX; quem autoriza é a API.	Cadastro público cria só PROFISSIONAL. ADMIN vem de seed ou de outro admin. Registrar a estratégia de token (Bearer de vida curta ou cookie httpOnly), isolada em um único módulo do front.	Antes do login (Sprint 1)

D8	§4	Confirmar	CPF e nº do Crea são dados pessoais (LGPD). Faltam obrigatoriedade, formato, validação e política de exibição.	Definir campos obrigatórios e validações. O FE mascara CPF em listagens e nunca o coloca em URL ou log.	Antes do cadastro
D9	§1, §3, §4	Problema real	Não há requisitos funcionais ou não funcionais nem critérios de aceite no PDF. `docs/` os menciona, mas não foram enviados. Só há "responsivo", sem navegadores, acessibilidade ou desempenho.	O PO escreve critérios de aceite por tela, começando por login e recursos. O FE propõe um checklist mínimo de requisitos não funcionais.	Mínimo antes da Sprint 1; o resto por sprint
D10	todo	Problema real	Não há contrato de API (endpoints, DTOs, formato de erro, paginação, OpenAPI). O front depende de tudo e a integração tenderia a ficar para o fim.	Contrato primeiro: OpenAPI/Swagger por módulo e formato único de erro, com revisão do FE. Mocks só se o back atrasar.	Auth e recurso antes da Sprint 1; demais antes das Sprints 2 e 3
D11	§2, §6	Problema real	Não há papel de UX/UI, wireframes, fluxos nem identidade visual. O calendário no celular é o ponto mais delicado.	PO e FE fazem wireframes de baixa fidelidade das telas do MVP, validados com a Mútua-PI. Usar uma lib de UI para reduzir design customizado.	Mínimo antes da Sprint 1; calendário antes da Sprint 2
D12	§3, §6	Problema real	`notificacao`, `relatorio` e `common` não têm dono. O "histórico por usuário e por recurso" (MVP) provavelmente depende de endpoint do `relatorio`.	Atribuir os módulos e definir os endpoints de histórico.	`common`: Sprint 1; histórico: antes da Sprint 3

D13	§5, §6	Problema real	Telas sem dono ou definição: login/cadastro/logout, layout/navegação, "cadastros" do admin (só recursos? usuários?), cancelar/alterar reserva (implícito em "minhas reservas") e conteúdo do painel do dia.	Montar uma tabela de telas do MVP com dono, perfil e rota.	Antes de codar telas
D14	§3	Melhoria	O back é organizado por funcionalidade para evitar conflitos, mas o front usa pastas por camada compartilhadas (`components/`, `services/`, `hooks/`). Com dois devs, isso gera conflitos de merge.	Dentro de `services/` e `components/`, criar subpastas por domínio (recurso, reserva, emprestimo) e uma `components/ui/` com dono único.	Na criação da estrutura
D15	§4	Confirmar	Transições do ciclo da reserva não definidas. Quem e como muda para EM_USO, CONCLUIDA e NAO_COMPARECEU (manual ou automático)? Salas passam por EM_USO, ou só equipamentos (já que `Emprestimo` cobre equipamentos)? Faltam prazos e quem pode cancelar/alterar. Para o BE: a constraint deve considerar só status ativos (CANCELADA e NAO_COMPARECEU liberam o horário).	Montar a matriz status × ação × perfil.	Cancelamento: Sprint 2; demais: Sprint 3

D16	§2, §8	Problema real	Docker, CI e deploy estão sem dono. O deploy só aparece na Sprint 4, sem destino. Rodar o front em Docker durante o desenvolvimento prejudica o hot reload.	Em dev, usar Vite local e API+banco no Compose (proxy/CORS). Dockerfile do front só para o build final. Definir dono e destino do deploy cedo. CI do front (lint, typecheck, test, build) desde a Sprint 1.	Dono e destino na Sprint 1
D17	§8	Melhoria	O cronograma não tem datas. Testes do front e validação com a Mútua-PI ficam só na Sprint 4, o que gera retrabalho tardio. A Sprint 1 é pesada para o FE (setup + auth + layout + CRUD) e o FE1 não tem entrega listada antes da Sprint 2.	Demo intermediária no fim da Sprint 2, testes contínuos nos fluxos críticos e redistribuição da fundação entre os FEs (se houver dois).	Planejamento de sprint
D18	§4	Melhoria	`Recurso` não tem categoria de equipamento (drone, pacômetro…), descrição, local ou imagem. Várias unidades do mesmo equipamento viram vários Recursos. Isso limita filtros e listagens.	Confirmar se nome + tipo bastam no MVP.	Antes do CRUD de recursos
D19	§7	Melhoria	A convenção cita só `feat/fix/docs`. `git add .` pode incluir arquivos indevidos. Proteção da `main` e templates de PR/Issue estão sem responsável.	Incluir `chore/refactor/test/style`, ter `.gitignore` correto e definir quem configura as proteções.	Depois (não bloqueia)

Parte 3 — Suas responsabilidades como Front-end

Tratei o §6 como divisão provisória. Marquei o que seria do **FE1** (profissional) e do **FE2** (admin). Se você for o único FE, tudo isso é seu. Se houver dois, só a sua metade.

A. Atribuídas explicitamente (§6)

ID	Tarefa	Resp. atual	Pré-requisitos	Critério de conclusão	Prior.	Quando
A1	Disponibilidade em calendário	FE1	Contrato de disponibilidade, D5, D6, wireframe, lib de calendário	O profissional escolhe um recurso e vê períodos livres e ocupados corretos, no celular e no desktop	P1	Sprint 2
A2	Criar reserva	FE1	A1, contrato de reserva, regras do MVP (D3)	Reserva criada. Em conflito, mostra mensagem clara e atualiza o calendário	P1	Sprint 2
A3	Minhas reservas: listar, cancelar, alterar (inferência: inclui histórico próprio)	FE1	A2, regras de cancelamento (D15)	Ações coerentes com o status, com confirmação antes de cancelar	P1	Sprints 2–3 (inferência)
A4	Cadastro de recursos: listar, criar, editar, ativar/desativar (Sprint 1 cita "cadastro de recursos"; "cadastros" pode incluir usuários, confirmar)	FE2	Base do front (B6–B12), contrato de recurso, D18	CRUD funcional, acessível só para ADMIN	P0	Sprint 1
A5	Painel admin com reservas do dia	FE2	Contrato de listagem por data, D13	Admin vê as reservas do dia, filtráveis	P1	Sprint 3

A6	Retirada e devolução com condição	FE2	Contrato de `emprestimo`, D4, D15	Registra retirada e devolução, e o status é atualizado	P1	Sprint 3
A7	Histórico por usuário e por recurso	FE2	Endpoints (D12)	Filtros por período e paginação	P2	Sprint 3

B. Necessárias e sem responsável (recomendações, não atribuições automáticas)

ID	Tarefa pendente (objetivo → conclusão)	Motivo	Resp. no doc	Resp. recomendado	Dependências	Prior.	Etapa
B1	Fechar stack e libs do FE (router, UI, dados, formulários, calendário, testes) → decisão registrada	Libs não citadas	Ninguém	FE propõe, grupo aprova	D1, D2	P0	E0
B2	Repo monorepo, `main`/`develop`, proteção da `main`, templates, quadro do Projects → repo pronto	Sprint 1 cita "Repositório" sem dono	Ninguém	PO/Docs ou dono do GitHub da equipe	—	P0	E0–E1
B3	Wireframes e fluxos de navegação do MVP → telas validadas	Sem UX/designer	Ninguém	PO/QA/Docs + FE; validação Mútua-PI	Levantamento	P0 (mínimo)	E0 e refino

B4	Matriz de perfis/permissões e critérios de aceite por tela → documento	Só há 2 perfis, sem regras	PO/QA/Docs (parcial: levantamento)	PO	Mútua-PI	P0	E0
B5	Contrato da API (OpenAPI + formato de erro) → publicado e revisado	Front depende dele	Ninguém	BE1 (auth/usuario/recurso), BE2 (reserva/regra/emprestimo); FE revisa	—	P0	E0 (auth/recurso); E4–E5
B6	Scaffold do front e estrutura de pastas → projeto builda	O doc mostra as pastas, não quem as cria	Ninguém	FE (um único dono)	B1, B2	P0	E1
B7	ESLint, Prettier, TS strict, testes e job de CI do front → CI verde	Padronização e proteção de PR	Ninguém	FE (job); workflow base de quem fizer o CI geral (sugestão: BE1)	B6	P1	E1
B8	Ambiente: `.env`/`VITE_API_URL`, proxy/CORS, Compose (API+DB), seed (admin inicial, recursos de exemplo)	Sprint 1 cita Docker sem dono	Ninguém	BE1 (Compose, CORS, seed); FE (`.env`, proxy, Dockerfile do front depois)	—	P0 (integração)	E1
B9	Rotas, guardas por perfil, layouts e navegação mobile/desktop	Estrutura não cobre	Ninguém	FE	B4, B6	P0	E2

B10	Camada de API, erros, tipos e utilitários de data/fuso	Sem padrão definido	Ninguém	FE com BE	B5	P0	E2
B11	Login/cadastro, sessão, logout e 401/403	Nenhum FE os lista	Ninguém (BE1 faz a API)	FE	B5, decisão de token	P0	E3
B12	Base visual e componentes compartilhados; estados de carregamento/erro/vazio; responsividade e acessibilidade	`components/` existe sem dono	Ninguém	FE (dono único de `ui/`)	B3	P1	E2 e contínuo
B13	Testes: unitários/componentes (FE) e integração FE↔BE/aceitação (QA)	Doc só cita "testes" para PO/QA e Sprint 4	PO/QA/Docs (parcial)	FE testa seus componentes; QA valida integração a cada sprint	B5	P1	Contínuo + E6
B14	`notificacao`, `relatorio`, `common` e endpoints de histórico	Sem dono; o histórico depende disso	Ninguém	Back-end (BE1: `common`; BE2 ou BE1: `relatorio`)	D12	P1	Sprint 1 (`common`); Sprint 3
B15	Destino do deploy e documentação técnica (README do front, decisões)	Sprint 4 sem dono nem destino	PO/Docs (README, parcial)	Deploy: BE1 + FE. Docs: PO/Docs com FE	—	P1 (destino), P2	E0 (destino); E6

C. Dependem de outras pessoas

ID	Entrega ou decisão necessária	De quem	O que bloqueia	Prior.	Prazo-limite
C1	Contrato de auth, decisão de token e CORS	BE1	B10, B11	P0	Antes da Sprint 1
C2	Contrato de recurso e seed	BE1	A4	P0	Sprint 1
C3	Contrato de disponibilidade, reserva e erros de conflito/regra	BE2	A1–A3	P0	Início da Sprint 2
C4	Contrato de `emprestimo` e histórico	BE2 (+ dono de `relatorio`)	A5–A7	P1	Início da Sprint 3
C5	Escopo do MVP fechado (D3, D4, D15)	PO + Mútua-PI	A2, A3, A6	P0	Antes da Sprint 2
C6	Coworking e granularidade/fuso de datas (D5, D6)	PO + BE2	A1, A2	P0	Antes da Sprint 2
C7	Wireframes e identidade visual	PO (+ Mútua-PI)	Todas as telas	P0	Mínimo antes da Sprint 1
C8	Matriz de permissões	PO	B9	P0	Antes da Sprint 1
C9	Docker/CI/destino de deploy	BE1 / grupo	Integração e E6	P1	Sprint 1

Parte 4 — Pendências anteriores à codificação

4.1 Antes de qualquer código (bloqueia o scaffold)

1. Confirmar seu papel e o time real (D1).
2. Confirmar a stack e as libs mínimas (D2/B1).
3. Garantir repositório, acesso, branches e Node/gerenciador de pacotes (B2).

Por que primeiro: são as decisões mais baratas de resolver agora e as mais caras de mudar depois.

4.2 Antes das primeiras telas (Sprint 1)

- Contrato de auth e recurso (C1, C2), estratégia de token (D7) e seed de admin.
- Matriz de permissões e tabela de telas do MVP (D13).
- Wireframes mínimos: login, cadastro, lista e formulário de recurso.

- Campos obrigatórios e validações do cadastro (D8).
- Escolha da lib de UI/estilo.

4.3 Antes da Sprint 2 (calendário e reserva)

- Coworking (D5), datas/fuso/granularidade (D6) e contrato de disponibilidade e conflito.
- Regras de uso do MVP e se a reserva nasce PENDENTE ou CONFIRMADA (D3).
- Lib de calendário e wireframe do calendário no celular.

4.4 Antes da Sprint 3

- Matriz de transições de status (D15), "condição" e fotos (D4), conteúdo do painel (D13) e dono do histórico (D12).

4.5 Pode ser simplificado ou resolvido durante o desenvolvimento

- **Estado:** Context só para auth, mais uma lib de dados do servidor. Sem Redux.
- **Tema:** tokens simples, refinados por sprint.
- **Testes:** foco nos fluxos críticos (login, guarda de rota, conflito de reserva, retirada/devolução).
- **Documentação:** README do front e decisões em `docs/`. Nada além disso é necessário no início.
- **Fora de escopo:** i18n, PWA e app nativo (Fase 2).

Parte 5 — Plano de execução priorizado

Tarefas bloqueadoras:

- **B1/B2/B6:** stack, repositório e scaffold. Sem eles nada começa.
- **B5/C1:** contrato e decisão de token, que bloqueiam login e todas as telas protegidas.
- **C3 + C6:** contrato de disponibilidade e decisão do coworking, que bloqueiam calendário e reserva.
- **C5:** escopo do MVP, que bloqueia reserva, cancelamento e retirada/devolução.

E0 — Alinhamentos e decisões (antes da Sprint 1) · *bloqueadora*

- **Fazer:** obter as respostas P0 da Parte 6 e registrá-las.
- **Por que primeiro:** stack e auth afetam toda a arquitetura.
- **Entrega:** decisões registradas, tabela de telas e wireframes mínimos.
- **Verificação:** o campo "decisões confirmadas" do BLOCO-BASE preenchido e nenhuma pergunta P0 aberta.
- **Em paralelo:** BE publica o contrato de auth e recurso, o PO escreve critérios de aceite, BE sobe Compose e banco.

E1 — Ambiente e scaffold (Sprint 1) · Prompts 0, 1, 2 · *bloqueadora*

- **Fazer:** inspecionar o repo, criar `frontend/`, estrutura, lint/format/testes, `.env` e CI do front.
- **Por que nessa ordem:** todo o resto depende de uma base que builda, com um único dono para evitar conflito.
- **Pré-requisitos:** E0 (papel, stack, repo).
- **Entrega:** projeto que roda e builda, com CI verde.
- **Verificação:** `lint`, `typecheck`, `test` e `build` passam, e o PR é aprovado por outro integrante.
- **Em paralelo:** Compose, banco e auth no back; plano de testes do QA.

E2 — Arquitetura e base visual (Sprint 1) · Prompts 3, 4, 5

- **Fazer:** rotas, layouts e guardas; cliente de API, erros e datas; tokens e componentes compartilhados; estados de carregamento, erro e vazio.
- **Por que nessa ordem:** vem antes das telas de negócio para evitar duplicação, e a guarda de rota vem antes das telas protegidas.
- **Pré-requisitos:** E1, matriz de permissões, libs, wireframes e contrato de auth.
- **Entrega:** app navegável com placeholders, cliente HTTP único e biblioteca `ui/`.
- **Verificação:** testes de guarda e de cliente; revisão em 375 px e 1280 px.
- **Em paralelo:** BE entrega auth e recurso. Se houver dois FEs, o segundo prepara wireframes e a avaliação da lib de calendário.

E3 — Autenticação e cadastro de recursos (Sprint 1) · Prompts 6, 7 · *auth bloqueia todas as telas protegidas*

- **Pré-requisitos:** C1, C2, decisão de token e seed de admin.
- **Entrega:** login, cadastro, logout e CRUD de recursos do admin.
- **Verificação:** fluxos funcionam contra a API real e só ADMIN acessa o admin.
- **Em paralelo:** BE2 inicia reserva; FE1 faz protótipo do calendário.

E4 — Disponibilidade, calendário e reserva (Sprint 2) · Prompts 8, 9, 10

- **Pré-requisitos:** D3, D5, D6 decididos; C3; recursos cadastrados.
- **Entrega:** calendário reutilizável, criação de reserva com tratamento de conflito e "minhas reservas".
- **Verificação:** conflito testado com duas contas tentando o mesmo horário; testes de componente; **demo para a Mútua-PI** (recomendação D17).
- **Em paralelo:** BE2 finaliza regras; QA escreve casos de conflito.

E5 — Painel, retirada/devolução e histórico (Sprint 3) · Prompts 11, 12, 13

- **Pré-requisitos:** D4, D12, D15; C4.
- **Entrega:** painel do dia, registro de retirada e devolução, histórico por usuário e por recurso.
- **Verificação:** ciclo completo (reserva → retirada → devolução) funcionando ponta a ponta.

E6 — Qualidade, deploy e documentação (Sprint 4, mas contínua) · Prompts 14, 15

- **Pré-requisitos:** funcionalidades completas e destino de deploy.
- **Entrega:** testes dos fluxos críticos, revisão de acessibilidade e responsividade, build de produção e README.
- **Verificação:** build servido corretamente, com rotas SPA e `VITE_API_URL` configurável.

Parte 6 — Perguntas para o alinhamento com a equipe

Time e infraestrutura

1. Quantos integrantes de fato somos, e eu sou FE1, FE2 ou os dois?
2. Confirmamos React + Vite + TS? Quais libs para rotas, UI/estilo, dados do servidor, formulários, calendário e testes?
3. O repositório já existe? Há código? Quem configura a proteção da `main`, templates de PR e o Projects?
4. Quem cria o Compose, o CI e o deploy? Onde será o deploy?
5. Quem faz os wireframes? Existe identidade visual da Mútua-PI ou do Crea Tec HUB? Quem valida o design?

Back-end

6. Haverá OpenAPI/Swagger? Quando auth e recurso estarão acessíveis em dev?
7. O token vai no header ou em cookie httpOnly? Qual a expiração, existe refresh e como o front obtém o usuário atual e o perfil?
8. Qual o formato padrão de erro (validação por campo, conflito, regra violada) e quais códigos HTTP são usados?
9. Datas: ISO 8601 com offset ou UTC? Qual o fuso do sistema?
10. Qual o endpoint de disponibilidade e o formato da resposta (reservas ocupadas ou horários livres)? Quais os padrões de paginação e filtros?
11. Como subir API e banco localmente? Há CORS/proxy e seed (admin inicial, recursos de exemplo)?
12. Quem faz `relatorio` (histórico por usuário e por recurso), `notificacao` e `common`?

PO / Mútua-PI

13. Quem pode se cadastrar? O nº do Crea é obrigatório e validado (e por quem)? Quem cria contas ADMIN?
14. A reserva nasce CONFIRMADA ou PENDENTE? Quais regras de uso entram no MVP (duração máxima, antecedência, horário de funcionamento)?
15. Coworking: reserva por posto, por capacidade ou o espaço inteiro?
16. Qual a granularidade de horário? Equipamentos são reservados por hora ou por dia(s)? Reservas podem atravessar dias?
17. Qual o prazo e quem pode cancelar/alterar? O admin pode reservar em nome de um profissional?
18. NAO_COMPARECEU é manual (admin) ou automático? Salas têm check-in/EM_USO ou só equipamentos?
19. "Condição" na retirada e devolução é texto livre ou lista fixa? Fotos ficam fora do MVP?

20. O que o painel "reservas do dia" deve mostrar? "Cadastros" do admin inclui gerenciar usuários?
21. Recuperação de senha precisa existir no MVP? Notificações por e-mail ficam realmente fora?
22. Quais navegadores e dispositivos mínimos devemos suportar? Haverá demo para a Mútua-PI antes da Sprint 4?

Parte 7 — Prompts prontos para a IA

Como usar: cole o **BLOCO-BASE** antes de cada prompt. Ele reúne o contexto do projeto (item 1), as restrições gerais (item 5) e o relatório final (item 9). Cada prompt traz os itens 2, 3, 4, 5 (específicas), 6, 7 e 8. Preencha o campo "DECISÕES CONFIRMADAS" conforme a equipe responder à Parte 6 e atualize-o a cada prompt.

BLOCO-BASE (cole antes de todo prompt)

1. CONTEXTO DO PROJETO
Fonte: documento "Estrutura do Projeto — Sistema de Reservas Mútua-PI" (ICEV, out/2026). É uma PROPOSTA; só trate como definido o que estiver em "DECISÕES CONFIRMADAS".
- Aplicação web responsiva de reservas de ambientes compartilhados (coworking, sala de estudos, salas multiuso, sala de reunião, auditório) e equipamentos do Crea Tec HUB, com controle de retirada e devolução. O profissional usa principalmente o celular (mobile-first); a equipe administrativa usa computador.
- Arquitetura: Front-end React + Vite + TypeScript → API REST/JSON Spring Boot (Java 21) → PostgreSQL. Autenticação JWT com perfis PROFISSIONAL e ADMIN.
- Monorepo mutua-reservas/: backend/, frontend/, docs/, docker-compose.yml, .github/workflows/. Estrutura prevista do front: frontend/src/pages/profissional, pages/admin, components, services, hooks.
- Entidades: Usuario, Recurso (tipo COWORKING/SALA/AUDITORIO/EQUIPAMENTO), RegraUso, Reserva (PENDENTE, CONFIRMADA, EM_USO, CONCLUIDA, CANCELADA, NAO_COMPARECEU), Emprestimo, Notificacao.
- Git: main protegida, develop, feature/nome-da-tarefa, commits feat:/fix:/docs:, ninguém faz merge do próprio PR.
DECISÕES CONFIRMADAS: [PREENCHER: papel do FE, libs escolhidas, contrato da API, token, coworking, formato de datas, escopo do MVP...]

5. RESTRIÇÕES GERAIS
- Antes de alterar qualquer coisa, inspecione a estrutura e os arquivos existentes. Preserve o que está correto; não recrie configurações ou funcionalidades já implementadas.
- Siga as convenções do projeto, reutilize componentes e funções existentes e evite duplicação.
- Não invente telas, endpoints, campos, regras de negócio ou tecnologias. Se faltar informação indispensável, PARE e liste como pendência ou pergunta; não escolha sozinho quando a decisão afetar arquitetura ou regra de negócio.

- Não modifique arquivos fora do escopo sem justificar. Não instale dependências desnecessárias (justifique cada uma). Não sobrescreva alterações existentes sem necessidade.
- Não implemente itens da Fase 2 (lembretes por e-mail/WhatsApp, fluxo de aprovação, relatórios com gráficos, fotos de condição, lista de espera, penalidades, app nativo), salvo se constar em DECISÕES CONFIRMADAS.
- Textos de interface em português do Brasil. Não considere a tarefa concluída sem verificar cada critério de aceite.

9. RELATÓRIO FINAL (obrigatório)
(1) arquivos criados/alterados; (2) decisões tomadas e justificativa; (3) comandos de validação executados e resultados; (4) cada critério de aceite: atendido / não atendido; (5) problemas, conflitos e impedimentos; (6) pendências e dúvidas para a equipe.

Prompt 0 — Investigar repositório e ambiente (somente leitura)

Pré-requisito: repositório clonado. **Resultado:** diagnóstico do que existe, para adaptar ou pular os prompts 1 a 5.

2. OBJETIVO: Mapear, SEM alterar nada, o estado atual do repositório e do ambiente.
3. ESCOPO: raiz do repositório; frontend/ (se existir); backend/ apenas leitura de controllers, DTOs e configuração de OpenAPI; docker-compose.yml; .github/workflows; README; docs/.
4. REQUISITOS: Informe (a) se frontend/ existe e o que contém (package.json, versões de Node/React/Vite/TS, libs, scripts, ESLint/Prettier/testes, .env*, estrutura comparada à prevista no documento); (b) como subir API e banco localmente (compose, portas, variáveis, CORS) e se há seed; (c) endpoints já existentes no back (método, rota, perfil exigido, DTOs) e se há OpenAPI/Swagger; (d) workflows de CI; (e) convenções de branch/commit observadas em git log; (f) divergências entre o código e o documento.
5. RESTRIÇÕES ESPECÍFICAS: somente leitura. Não instale dependências, não crie/edite/apague arquivos, não rode comandos que modifiquem o repositório.
6. DEPENDÊNCIAS: repositório clonado localmente.
7. CRITÉRIOS DE ACEITE: relatório por item (a)–(f) com "encontrado / não encontrado / não consegui verificar"; lista de lacunas; recomendação de quais prompts seguintes podem ser pulados ou adaptados.
8. VALIDAÇÃO: liste os comandos de leitura usados e o que não pôde ser verificado.

Prompt 1 — Scaffold do front-end

Pré-requisito: Prompt 0 verificado; stack confirmada; branch `feature/...` criada a partir da `develop`. **Resultado:** projeto que roda e builda.

2. OBJETIVO: Criar (ou completar, se já existir) o projeto React + Vite + TypeScript em frontend/, rodando e buildando, com a estrutura de pastas do documento.

3. ESCOPO: frontend/package.json, vite.config.ts, tsconfig*, index.html, src/main.tsx, src/App.tsx; pastas src/pages/profissional, src/pages/admin, src/components, src/services, src/hooks (e src/types e src/utils apenas se necessário); .env.example; .gitignore; frontend/README.md.
4. REQUISITOS: TypeScript em modo strict; gerenciador de pacotes e versão de Node conforme DECISÕES CONFIRMADAS (se ausentes, pergunte antes); scripts dev, build, preview; .env.example com VITE_API_URL; página inicial placeholder; README curto com como instalar, rodar e buildar. Dentro de services/ e components/ prefira subpastas por domínio (recurso, reserva, emprestimo) e components/ui/ para itens compartilhados.
5. RESTRIÇÕES ESPECÍFICAS: não instalar router, lib de UI nem de estado neste prompt; não criar telas de negócio; não alterar backend/, docker-compose.yml nem workflows.
6. DEPENDÊNCIAS: Prompt 0 concluído; D2 e B2 resolvidos (stack e repo).
7. CRITÉRIOS DE ACEITE: install, dev e build funcionam sem erros; tsc sem erros; estrutura de pastas criada; .env real ignorado pelo git; README claro.
8. VALIDAÇÃO: rodar install, tsc --noEmit, build e dev (confirmar que a página carrega).

Prompt 2 — Qualidade: lint, formatação, testes e CI do front

Pré-requisito: Prompt 1 funcionando. **Resultado:** `lint`, `typecheck`, `test` e `build` passam localmente e no CI.

2. OBJETIVO: Padronizar a qualidade do código do front-end e automatizá-la.
3. ESCOPO: configs de ESLint e Prettier, Vitest + Testing Library (jsdom), scripts no package.json, um teste de fumaça do App e, se a equipe confirmar, um job de CI para frontend/ em .github/workflows.
4. REQUISITOS: scripts lint, format, typecheck, test, build; regras de ESLint compatíveis com TS strict e React; um teste de fumaça que renderiza o App; job de CI executando install, lint, typecheck, test e build apenas para mudanças em frontend/, em PRs para develop e main.
5. RESTRIÇÕES ESPECÍFICAS: não reformatar arquivos fora de frontend/; não alterar workflows do back-end; não adicionar hooks de commit (husky/commitlint) sem confirmação.
6. DEPENDÊNCIAS: Prompt 1; B2 (repositório) e decisão sobre quem mantém o CI.
7. CRITÉRIOS DE ACEITE: os quatro scripts passam; o teste de fumaça passa; o CI (se criado) é válido e passa.
8. VALIDAÇÃO: executar todos os scripts e mostrar as saídas.

Prompt 3 — Rotas, layouts e guardas por perfil

Pré-requisito: Prompt 2; matriz de permissões e tabela de telas do MVP. **Resultado:** app navegável com placeholders e proteção por perfil.

2. OBJETIVO: Implementar a navegação do app: rotas, layouts e proteção por perfil, usando um AuthContext simulado (mock).
3. ESCOPO: instalação do roteador confirmado; src/routes; layouts (público para login/cadastro; profissional com navegação adequada ao celular; admin com navegação

para desktop); páginas placeholder; páginas de não encontrado (404) e acesso negado; guarda de rota por perfil; interface do AuthContext.
4. REQUISITOS: rotas e nomes conforme a tabela de telas do MVP confirmada (se ausente, PARE e pergunte); não autenticado → login; perfil sem permissão → acesso negado; regra de acesso do ADMIN às rotas de PROFISSIONAL conforme a matriz confirmada; layouts responsivos. Lembre no código que a guarda é conveniência de UX; a autorização real é da API.
5. RESTRIÇÕES ESPECÍFICAS: não implementar login real nem escolher forma de armazenar token; não criar telas finais.
6. DEPENDÊNCIAS: Prompt 2; matriz de permissões; lista de telas e rotas.
7. CRITÉRIOS DE ACEITE: navegação entre placeholders; guarda redireciona corretamente para sessão sem login, PROFISSIONAL e ADMIN simulados; rota inexistente mostra 404; menu utilizável em 375 px e 1280 px.
8. VALIDAÇÃO: testes de componente da guarda (no mínimo 3 cenários); lint, typecheck, test, build; checagem manual nas duas larguras.

Prompt 4 — Camada de API, erros e datas

Pré-requisito: Prompt 3; contrato OpenAPI de auth e recurso (mínimo); decisão de token; decisão de formato de datas. **Resultado:** um único ponto de chamada HTTP.

2. OBJETIVO: Criar a camada central de acesso à API, com tratamento padronizado de erros e utilitários de data.
3. ESCOPO: src/services (cliente HTTP e módulos por domínio apenas para endpoints CONFIRMADOS), src/types, src/utils/date.
4. REQUISITOS: baseURL vinda de VITE_API_URL; credencial anexada conforme a estratégia de autenticação confirmada (se não confirmada, deixe um ponto de extensão e liste como pendência); normalizar erros para um tipo único conforme o formato confirmado do back; tratar 401/403 de forma centralizada (gancho para logout/redirecionamento); tipos TypeScript derivados do contrato OpenAPI quando existir, senão apenas dos endpoints confirmados; utilitários de data conforme formato e fuso confirmados (se não definidos, PARE e liste como pendência).
5. RESTRIÇÕES ESPECÍFICAS: não invente endpoints; não faça chamadas HTTP dentro de componentes; use fetch nativo ou a lib confirmada; não instale libs de data sem justificativa.
6. DEPENDÊNCIAS: Prompt 3; contratos mínimos; decisões de token e datas.
7. CRITÉRIOS DE ACEITE: um único ponto de chamada HTTP; erros normalizados; 401 tratado centralmente; testes unitários cobrindo sucesso, 401, erro de validação e falha de rede.
8. VALIDAÇÃO: testes com mocks; typecheck; se a API local estiver disponível, chamar um endpoint real confirmado e reportar o resultado.

Prompt 5 — Base visual e componentes compartilhados

Pré-requisito: Prompt 3; lib de UI/estilo decidida; wireframes mínimos. **Resultado:** biblioteca `ui/` reutilizável.

2. OBJETIVO: Criar a base visual e os componentes compartilhados que as telas vão reutilizar.
3. ESCOPO: tokens (cores, tipografia, espaçamento, breakpoints mobile-first); src/components/ui: Button, Field/Input (com erro), Select, Modal/Dialog, Card, Table responsiva; estados padrão Loading, ErrorState (com "tentar de novo"), EmptyState; Toast/Alert; rota de demonstração apenas em desenvolvimento.
4. REQUISITOS: seguir wireframes e identidade confirmados (se não houver identidade, usar paleta neutra com tokens centralizados para troca fácil); acessibilidade básica: labels associados, foco visível, contraste adequado, navegação por teclado, aria no Modal, alvos de toque adequados no celular; status nunca indicado só por cor.
5. RESTRIÇÕES ESPECÍFICAS: não criar componentes de domínio (calendário, cartão de reserva); não instalar libs de UI além da confirmada.
6. DEPENDÊNCIAS: Prompt 3; decisão da lib de UI; wireframes mínimos.
7. CRITÉRIOS DE ACEITE: todos os componentes visíveis na rota de demonstração; Modal e Field operáveis por teclado; tokens centralizados; responsivo em 375, 768 e 1280 px.
8. VALIDAÇÃO: testes de renderização e interação (Modal, Field); lint, typecheck, test, build; checagem manual de contraste e das três larguras.

Prompt 6 — Autenticação: login, cadastro e sessão

Pré-requisito: Prompt 4; contrato de auth; token decidido; seed de admin. **Resultado:** login funcionando contra a API real.

2. OBJETIVO: Implementar login, cadastro de profissional, sessão e logout.
3. ESCOPO: pages de login e cadastro; AuthContext real (substitui o mock do Prompt 3); services/auth; persistência de sessão conforme a estratégia confirmada; logout; tratamento de sessão expirada.
4. REQUISITOS: campos do cadastro conforme o contrato (o documento cita nome, e-mail, CPF e nº de registro no Crea; obrigatoriedade, máscara e validação conforme confirmado, senão PARE); cadastro público cria apenas PROFISSIONAL (salvo decisão contrária confirmada); após o login, redirecionar conforme o perfil; erros da API exibidos de forma segura; botão com estado de carregamento e sem duplo envio; nunca registrar senha ou CPF em console/log, nem colocar CPF em URL.
5. RESTRIÇÕES ESPECÍFICAS: não implementar recuperação de senha nem cadastro público de ADMIN; não armazenar senha.
6. DEPENDÊNCIAS: Prompt 4; contrato de auth (login, cadastro, usuário atual); decisão de token; seed de admin.
7. CRITÉRIOS DE ACEITE: login válido leva à rota correta por perfil; credencial inválida mostra mensagem; sessão expirada volta ao login; recarregar a página mantém a sessão conforme a estratégia; campos obrigatórios validados.
8. VALIDAÇÃO: testes de componente e de fluxo com mock; teste manual contra a API local; relatar resultados.

Prompt 7 — Admin: cadastro de recursos

Pré-requisito: Prompts 5 e 6; contrato de recurso; D18 resolvida. **Resultado:** CRUD de recursos para ADMIN.

2. OBJETIVO: Implementar a tela do admin para gerenciar ambientes e equipamentos (Recurso).
3. ESCOPO: src/pages/admin/recursos (lista, criar, editar, ativar/desativar), formulário de recurso, filtro por tipo.
4. REQUISITOS: campos do documento: nome, tipo (COWORKING, SALA, AUDITORIO, EQUIPAMENTO), capacidade, nº de patrimônio, ativo; campos de RegraUso (duração máxima, antecedência mín/máx, horário de funcionamento, exige aprovação) SOMENTE se confirmados no MVP; obrigatoriedade do nº de patrimônio conforme regra confirmada; confirmação antes de desativar; estados de carregamento/erro/vazio; paginação se o contrato definir; desktop-first, utilizável em tablet.
5. RESTRIÇÕES ESPECÍFICAS: não criar categoria, foto ou descrição sem constar no contrato; não implementar exclusão se o contrato não a prever.
6. DEPENDÊNCIAS: Prompts 5 e 6; contrato de recurso; D3 e D18 resolvidas.
7. CRITÉRIOS DE ACEITE: criar, editar e desativar refletem na lista; erros da API aparecem no campo ou como mensagem geral; PROFISSIONAL não acessa a tela.
8. VALIDAÇÃO: testes de formulário e de lista; teste manual contra a API; lint, typecheck, test, build.

Prompt 8 — Profissional: disponibilidade em calendário

Pré-requisito: Prompt 5; contrato de disponibilidade; coworking e datas decididos; lib de calendário escolhida. **Resultado:** consulta de disponibilidade.

2. OBJETIVO: Implementar a consulta de disponibilidade de recursos em calendário para o profissional.
3. ESCOPO: src/pages/profissional/disponibilidade; src/components/calendario (componente reutilizável); lista de recursos ativos com filtro por tipo; seleção de recurso e período.
4. REQUISITOS: exibir períodos livres e ocupados exatamente conforme a resposta do endpoint de disponibilidade confirmado; datas e fuso conforme o utilitário de datas; mobile-first (visão de dia ou lista no celular; semana/mês no desktop); exibir horário de funcionamento e regras apenas se vierem da API; estados de carregamento, erro e vazio; tratamento do coworking conforme decisão (se pendente, PARE).
5. RESTRIÇÕES ESPECÍFICAS: não calcular disponibilidade no front com regras próprias; não criar endpoint; não implementar a reserva aqui (apenas navegar para a rota de reserva com recurso e horário como parâmetros).
6. DEPENDÊNCIAS: Prompt 5; contrato de disponibilidade; D5 e D6; wireframe do calendário.
7. CRITÉRIOS DE ACEITE: períodos corretos para um recurso de teste; troca de recurso e de período atualiza a visualização; legível e operável em 375 px e 1280 px; erros de rede tratados.
8. VALIDAÇÃO: testes do componente de calendário com dados simulados; teste manual contra a API; lint, typecheck, test, build.

Prompt 9 — Profissional: criar reserva e tratar conflito

Pré-requisito: Prompt 8; contrato de reserva e de erros; D3 decidida. **Resultado:** fluxo de reserva completo.

2. OBJETIVO: Implementar a criação de reserva, com tratamento claro de conflito e de violação de regra.
3. ESCOPO: src/pages/profissional/reservar (página ou modal); formulário de recurso, início e fim (campos adicionais apenas se o contrato os tiver); integração com services/reserva; atualização do calendário após o resultado.
4. REQUISITOS: validações de interface apenas como feedback imediato (a API é a fonte da verdade); em conflito, mostrar mensagem clara e recarregar a disponibilidade; impedir duplo envio; estado exibido após a criação conforme o ciclo de vida confirmado (não presumir PENDENTE ou CONFIRMADA); mostrar mensagens de regra violada vindas da API.
5. RESTRIÇÕES ESPECÍFICAS: não implementar fluxo de aprovação; não presumir códigos HTTP ou formatos de erro não documentados.
6. DEPENDÊNCIAS: Prompt 8; contrato de reserva e de erros; D3.
7. CRITÉRIOS DE ACEITE: reserva válida é criada e aparece no calendário; conflito (inclusive simulado com duas contas) mostra mensagem e atualiza; duplo clique não duplica a reserva.
8. VALIDAÇÃO: testes de formulário e dos caminhos de erro; teste manual de conflito com duas contas; lint, typecheck, test, build.

Prompt 10 — Profissional: minhas reservas, cancelar e alterar

Pré-requisito: Prompt 9; regras de cancelamento/alteração e matriz de status (D15). **Resultado:** gestão das próprias reservas.

2. OBJETIVO: Implementar a listagem das reservas do profissional, com cancelamento, alteração e histórico próprio.
3. ESCOPO: src/pages/profissional/minhas-reservas; lista com status; filtros por status e período; detalhe; ações de cancelar e alterar.
4. REQUISITOS: ações disponíveis conforme o status e as regras de cancelamento/alteração confirmadas (prazos, quem pode); confirmação antes de cancelar; rótulo de status em texto, além de cor; paginação conforme o contrato; estados de carregamento, erro e vazio; alteração reutiliza o fluxo e as validações do Prompt 9.
5. RESTRIÇÕES ESPECÍFICAS: não presumir prazos ou regras; não permitir ações fora da matriz de transições confirmada.
6. DEPENDÊNCIAS: Prompt 9; contrato de listagem, cancelamento e alteração; D15.
7. CRITÉRIOS DE ACEITE: lista correta do usuário logado; cancelar e alterar funcionam e refletem na lista e no calendário; ações indevidas ficam ocultas ou desabilitadas conforme o status.
8. VALIDAÇÃO: testes de componente por status; teste manual; lint, typecheck, test, build.

Prompt 11 — Admin: painel com as reservas do dia

Pré-requisito: Prompt 5; conteúdo do painel definido (D13); contrato de listagem por data. **Resultado:** visão operacional do dia.

2. OBJETIVO: Implementar o painel do admin com as reservas do dia.
3. ESCOPO: src/pages/admin/painel; lista (ou agenda) do dia; filtros por recurso, tipo e status; atalhos para as ações de retirada e devolução (Prompt 12).
4. REQUISITOS: "dia" conforme o fuso confirmado; atualização manual por botão (sem polling ou websocket); desktop-first; estados de carregamento, erro e vazio; exibir apenas os campos que o contrato fornece.
5. RESTRIÇÕES ESPECÍFICAS: não incluir gráficos nem indicadores (Fase 2); não criar endpoint.
6. DEPENDÊNCIAS: Prompts 5 e 6; D13; contrato de listagem por data.
7. CRITÉRIOS DE ACEITE: mostra as reservas do dia corretas para a data; filtros funcionam; somente ADMIN acessa.
8. VALIDAÇÃO: testes de componente; teste manual contra a API; lint, typecheck, test, build.

Prompt 12 — Admin: retirada e devolução

Pré-requisito: Prompt 11; contrato de `emprestimo`; D4 e D15 decididas. **Resultado:** registro de retirada e devolução.

2. OBJETIVO: Implementar o registro de retirada e devolução de equipamentos, com a condição do equipamento.
3. ESCOPO: src/pages/admin/retiradas (ou fluxo dentro do painel); formulário de retirada (data e condição) e de devolução (data, condição e observações); integração com services/emprestimo.
4. REQUISITOS: "condição" como texto livre ou lista fixa conforme o contrato; o admin responsável vem da sessão ou do contrato, não de campo digitável; refletir a mudança de status (EM_USO e CONCLUIDA) conforme a resposta da API; bloquear ações inválidas pelo status; ação de marcar NAO_COMPARECEU apenas se confirmada como manual.
5. RESTRIÇÕES ESPECÍFICAS: sem upload de fotos (Fase 2, salvo decisão confirmada); não presumir que salas têm retirada/devolução.
6. DEPENDÊNCIAS: Prompt 11; contrato de emprestimo; D4 e D15.
7. CRITÉRIOS DE ACEITE: retirada e devolução registradas e visíveis no painel e no histórico; status atualizado corretamente; erros da API exibidos.
8. VALIDAÇÃO: testes dos formulários e dos estados bloqueados; teste manual do ciclo reserva → retirada → devolução; lint, typecheck, test, build.

Prompt 13 — Admin: histórico por usuário e por recurso

Pré-requisito: Prompt 12; endpoints de histórico existentes (D12); regras de exibição de dados pessoais. **Resultado:** consulta de histórico.

2. OBJETIVO: Implementar a consulta de histórico de reservas por usuário e por recurso.

3. ESCOPO: src/pages/admin/historico; seleção ou busca de usuário e de recurso; filtro por período; lista paginada; detalhe com dados de retirada e devolução quando existirem.
4. REQUISITOS: campos e filtros conforme o contrato; dados pessoais mínimos e CPF mascarado conforme a decisão confirmada; estados de carregamento, erro e vazio; desktop-first.
5. RESTRIÇÕES ESPECÍFICAS: sem gráficos nem indicadores de ocupação (Fase 2); não criar endpoint.
6. DEPENDÊNCIAS: Prompts 5, 6 e 12; contrato de histórico; D12.
7. CRITÉRIOS DE ACEITE: o histórico retornado corresponde ao usuário ou recurso selecionado; filtro por período e paginação funcionam; somente ADMIN acessa.
8. VALIDAÇÃO: testes de filtros e de paginação; teste manual; lint, typecheck, test, build.

Prompt 14 — Auditoria final: testes, acessibilidade e responsividade

Pré-requisito: funcionalidades do MVP implementadas. **Resultado:** lacunas fechadas, sem novas funcionalidades.

2. OBJETIVO: Revisar a qualidade do front-end e fechar lacunas, sem criar novas funcionalidades.
3. ESCOPO: todo o frontend/.
4. REQUISITOS: garantir testes dos fluxos críticos (login, guarda de rota, reserva com conflito, cancelamento, retirada e devolução); revisar acessibilidade (labels, foco, teclado, contraste, mensagens de erro); revisar responsividade em 375, 768 e 1280 px; revisar estados de carregamento, erro e vazio em todas as telas; remover console.log, dependências e código não usados; verificar que nenhum dado sensível é exposto em URL, log ou armazenamento além do definido.
5. RESTRIÇÕES ESPECÍFICAS: não refatorar em larga escala; corrija apenas defeitos pequenos e liste o restante como pendência.
6. DEPENDÊNCIAS: Prompts 6 a 13 concluídos.
7. CRITÉRIOS DE ACEITE: lint, typecheck, test e build passam; fluxos críticos cobertos; relatório de acessibilidade e responsividade com itens corrigidos e pendentes.
8. VALIDAÇÃO: executar todos os scripts e listar o que foi verificado manualmente.

Prompt 15 — Dockerfile do front, build de produção e documentação

Pré-requisito: destino de deploy definido; Compose mantido por outro integrante. **Resultado:** build de produção e README.

2. OBJETIVO: Preparar o build de produção do front-end e a documentação técnica.
3. ESCOPO: frontend/Dockerfile (build e servidor estático); frontend/README.md; registro das decisões do front em docs/ (arquivo próprio); proposta de trecho para o docker-compose.yml, sem aplicá-la sem alinhamento.
4. REQUISITOS: build recebe VITE_API_URL por variável; servidor com fallback para index.html nas rotas do SPA; nenhum segredo na imagem ou no repositório; README com instalação, scripts, variáveis, estrutura de pastas, convenções (commits e branches) e integração com a API.

5. RESTRIÇÕES ESPECÍFICAS: não alterar docker-compose.yml nem workflows de outros módulos sem justificativa e alinhamento; não definir infraestrutura de deploy não confirmada.
6. DEPENDÊNCIAS: Prompt 14; destino de deploy; Compose atual.
7. CRITÉRIOS DE ACEITE: imagem builda e serve o app; navegar direto para uma rota interna não retorna 404; README permite a um novo integrante rodar o projeto do zero.
8. VALIDAÇÃO: build da imagem, execução do container e acesso a uma rota interna; seguir o README em um diretório limpo.

Parte 8 — Checklist de acompanhamento

Legenda: `[ ]` pendente · `[~]` em andamento · `[x]` concluído · `[!]` bloqueado (anote o motivo).

E0 — Alinhamentos

- [ ] Papel (FE1/FE2/ambos) e time real confirmados
- [ ] Stack e libs do front aprovadas
- [ ] Repo, branches e proteção da `main` prontos
- [ ] Estratégia de token/sessão definida
- [ ] Matriz de perfis e permissões
- [ ] Tabela de telas do MVP e wireframes mínimos
- [ ] Contrato (OpenAPI) de auth e recurso publicado
- [ ] Escopo do MVP fechado (aprovação, notificações, fotos, regras)

E1 — Ambiente e scaffold

- [ ] Prompt 0 (diagnóstico)
- [ ] Prompt 1 (scaffold)
- [ ] Prompt 2 (qualidade e CI do front)

E2 — Arquitetura e base visual

- [ ] Prompt 3 (rotas, layouts, guardas)
- [ ] Prompt 4 (API, erros, datas)
- [ ] Prompt 5 (componentes `ui/`)

E3 — Auth e recursos

- [ ] Prompt 6 (login, cadastro, sessão)
- [ ] Prompt 7 (CRUD de recursos)

Decisões para a Sprint 2

- [ ] Coworking
- [ ] Datas, fuso e granularidade
- [ ] Contrato de disponibilidade, reserva e erros

E4 — Reserva

- [ ] Prompt 8 (disponibilidade)
- [ ] Prompt 9 (criar reserva e conflito)
- [ ] Prompt 10 (minhas reservas)
- [ ] Demo intermediária com a Mútua-PI

Decisões para a Sprint 3

- [ ] Matriz de transições de status
- [ ] Condição e fotos
- [ ] Conteúdo do painel
- [ ] Dono do histórico

E5 — Operação do admin

- [ ] Prompt 11 (painel do dia)
- [ ] Prompt 12 (retirada e devolução)
- [ ] Prompt 13 (histórico)

E6 — Fechamento

- [ ] Prompt 14 (auditoria)
- [ ] Prompt 15 (Docker, build, README)
- [ ] Deploy realizado

Contínuo, a cada tarefa

- [ ] Uma Issue por tarefa, em branch `feature/...`
- [ ] PR revisado por outro integrante
- [ ] Critérios de aceite verificados
- [ ] "Decisões confirmadas" do BLOCO-BASE atualizadas

Se quiser, coloco tudo isso em um documento editável para você acompanhar o checklist e copiar os prompts com mais conforto.